

Nama: Dhiya Dzakira Annuha
Kelompok: Vanadium

Problem 1 (pseudocode)

```
BEGIN

NUMBER N, K
INPUT N
INPUT K

NUMBER ARRAY astronot = [1, 2, 3, ..., N]
NUMBER ARRAY daftar_eliminasasi = []
indeks_sekarang = 0
indeks_daftelim = 0

WHILE size(astronot) > 1:
    NUMBER sisa = size(astronot)
    indeks_elim = (indeks_sekarang + K - 1) mod sisa
    NUMBER nomor_elim = astronot[indeks_elim]
    ADD nomor_elim TO daftar_eliminasasi[indeks_daftelim]
    indeks_daftelim = indeks_daftelim + 1
    REMOVE astronot[indeks_elim]

    indeks_sekarang = indeks_elim
    IF indeks_sekarang >= size_astronot
        indeks_sekarang = 0

    IF K < 2:
        K = 2
    ELSE IF nomor_elim mod 2 == 0:
        K = K + 2
    ELSE:
        K = K - 1

last_astro = astronot[0]
OUTPUT daftar_eliminasasi, last_astro

END
```

Penjelasan logika:

Permasalahan ini merupakan salah satu varian dari algoritma *Josephus Problem*, yakni sebuah algoritma perhitungan yang digunakan untuk memilih satu per satu orang dalam kelompok untuk dikeluarkan dari kelompok tersebut. Pada *Josephus Problem*, tiap orang dieliminasi tiap hitungan kelipatan K hingga tersisa satu orang pemenang yang tidak dieliminasi.

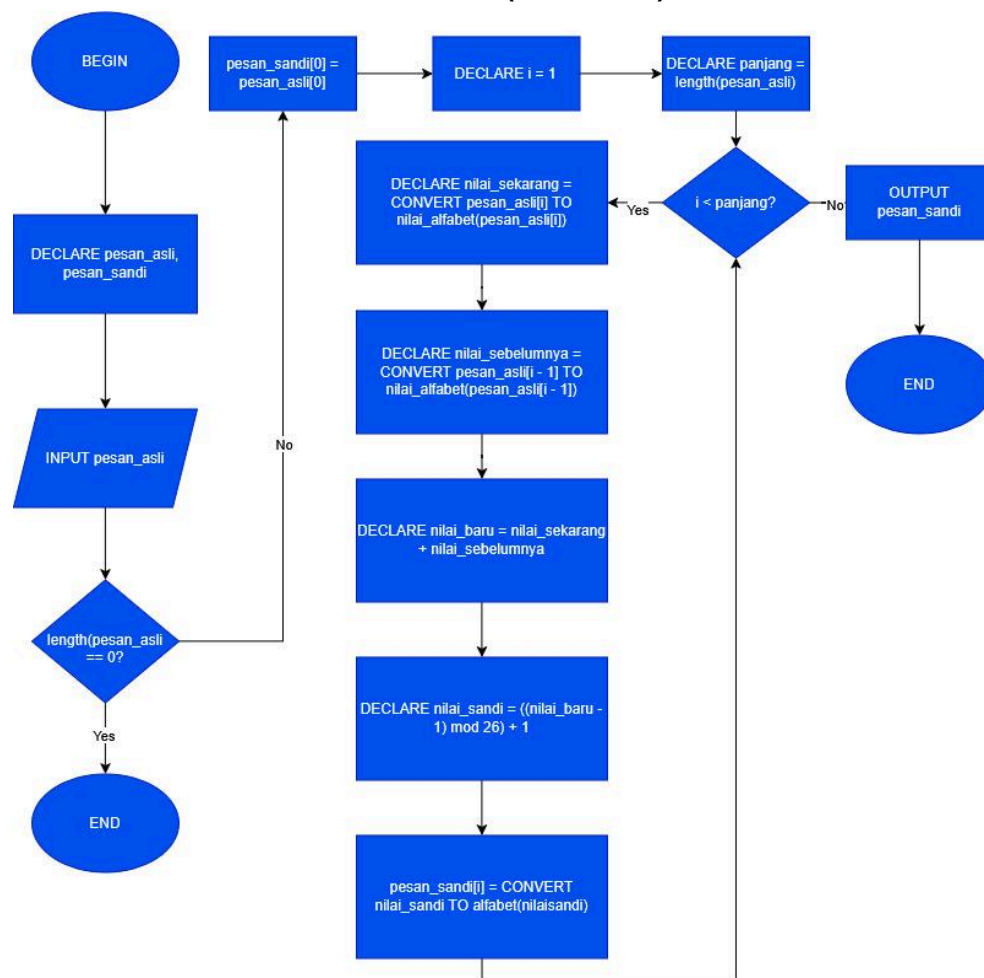
Namun, yang membedakan permasalahan ini dari *Josephus Problem* klasik adalah aturan bahwa nilai K dapat berubah tiap ada orang yang dieliminasi. Dalam penyelesaiannya secara pemrograman, konsep array dan *for loop* digunakan untuk menyimpan urutan orang yang dieliminasi dalam kelompok serta algoritma berulang untuk mengeliminasi mereka.

Terdapat satu rumus yang digunakan untuk memilih orang yang dieliminasi dalam *Josephus Problem*, yakni

$\text{Indeks_elim} = (\text{Indeks_sekarang} + K - 1) \bmod (\text{jumlah orang yang belum dieliminasi})$

dengan Indeks_elim adalah indeks urutan array untuk orang yang akan dieliminasi dan Indeks_sekarang adalah indeks urutan array untuk tiap fase perulangan. Modulo digunakan untuk memastikan bahwa perulangan dapat berputar kembali ke awal saat mencapai batas akhir urutan. Setelah orang yang akan dieliminasi didapat, orang tersebut dapat dipindahkan ke array khusus orang-orang yang tereliminasi, sehingga kita bisa mendapatkan urutan orang-orang yang tereliminasi sampai dengan perulangan terakhir, beserta orang terakhir yang tidak tereliminasi.

Problem 2 (flowchart)



Penjelasan logika:

Permasalahan ini merupakan jenis permasalahan kriptografi/sandi, di mana tiap huruf dalam sebuah kata atau kalimat harus diinterpretasikan dalam bentuk lain sesuai rumus yang berlaku agar dapat menjadi sebuah sandi. Sistem kriptografi juga dapat melakukan sebaliknya, yakni menginterpretasikan sebuah sandi menjadi bentuk yang dapat dimengerti, tetapi permasalahan ini meminta secara khusus untuk mengubah sebuah kata menjadi sandi sesuai aturan rumus yang berlaku. Aturan tersebut yakni bahwa untuk huruf kedua hingga terakhir pada kata akan digeser sebanyak nilai huruf yang berada tepat sebelumnya. (Contoh nilai huruf yakni A=1, B=2, C=3, dst.)

Cara menyelesaikan permasalahan ini adalah dengan mengonversi huruf yang akan diubah beserta huruf sebelumnya menjadi nilai hurufnya masing-masing, lalu menambahkan nilai huruf saat ini dengan nilai huruf sebelumnya, kemudian mengonversi kembali nilai tersebut menjadi bentuk huruf. Cara ini dilakukan secara satu persatu dan berulang hingga perulangan selesai mengubah huruf terakhir.